SAT Solver in Coq

Daniel Chitoraga
S-number: 5770777

November 7, 2025

1 Introduction

This project implements a Boolean Satisfiability (SAT) solver in the Coq proof assistant and formally proves its soundness. An SAT solver determines whether a propositional logic formula admits at least one valuation (truth assignment to variables) that makes it true.

The formulas are composed of propositional variables, Boolean constants ($\top$, $\bot$), and logical connectives $\wedge$, $\vee$, $\rightarrow$, and $\neg$. The goal is to define these formulas in Coq, specify their semantics, and implement an algorithm that decides the satisfiability.

As extensions, I implemented an *optimizer* that simplifies Boolean formulas using basic logical equivalences (Extension 1), and an *nnf_solver* that transforms Boolean formulas in negation normal form (Extension 2). As part of the extensions, I proved that both optimizations and conversion preserve satisfiability.

2 Representation of Boolean Formulas

The representation of Boolean formulas in Coq follows the grammar directly using an inductive type:

```
Inductive form :=
| FVar   : string -> form
| FTrue  : form
| FFalse : form
| FAnd   : form -> form -> form
| FOr    : form -> form -> form
| FImp   : form -> form -> form
| FNot   : form -> form.
```

This definition is straightforward and corresponds directly to the syntax of propositional logic. Using `string` for variable names keeps the system open-ended, allowing arbitrary propositional variables without a predefined finite set.

3 Semantics and Satisfiability

A *valuation* is a function mapping each variable name to a Boolean value:
`valuation` := `string` $\rightarrow$ `bool`.

The truth semantics of the formulas are defined recursively:

```
Fixpoint valid (V : valuation) (p : form) : Prop :=
match p with
| FVar x    => V x = true
| FTrue     => True
| FFalse    => False
| FAnd x y  => valid V x /\ valid V y
| FOr x y   => valid V x \/ valid V y
| FImp x y  => valid V x -> valid V y
| FNot x    => ~(valid V x)
end.
```

A formula is said to be *satisfiable* if there exists a valuation that makes it true: `satisfiable` $p := \exists V$, `valid` V p.

I verified this definition by testing it on example formulas such as $(x \vee \neg y) \wedge (\neg x \vee y)$ and $\neg y \to (x \vee y)$, showing that suitable valuations exist.

4 Enumerating Valuations and Implementing the Solver

The solver searches for a satisfying valuation by enumerating all possible truth assignments. The design proceeds in four steps:

1) **Collect variables:** `find_vars` traverses a formula and gathers all variable names.

2) **Generate truth assignments:** `gen_bool_perms` recursively constructs all Boolean combinations for the given number of (unique) variables.

3) **Generate valuations:** `gen_valuations := map (lookup vars) bool_perms.` Here `lookup: list string -> list bool -> string -> bool` returns the boolean value associated with a variable from a list of variables for a boolean permutation. Using partial application of `lookup`, it is mapped over the list of boolean permutations to create the the list of valuations : `list (string -> bool)`.

4) **Evaluate the formula:** `eval_form` computes the Boolean truth value of a formula under a given valuation. It is quite similar to `valid`, but instead of using `Prop` operators it uses `bool` operators.

The function that searches for a satisfying valuation is:

```
Definition find_valuation (p : form) : option valuation :=
  let vars := find_vars p in
  let bool_perms := gen_bool_perms (length vars) in
  find (fun V => eval_form V p) (gen_valuations vars bool_perms).
```

5 Soundness Proof

In order to prove the soundness of the solver (`solver p = true -> satisfiable p`), an induction was done on the list of all possible valuations. However, in this step, the lemma `eval_from_valid` was needed to complete the proof, because the induction hypothesis included `valid v p`, while another hypothesis stated that `eval_form v p = true`.

```
Lemma eval_form_valid :
  forall V p, eval_form V p = true <-> valid V p.
```

6 Extension 1: Optimizer

The first extension implements an **optimizer** for Boolean formulas. Its purpose is to simplify conjunctions and disjunctions using basic logical equivalences that preserve truth values:

$$p \wedge \top \equiv p, \qquad p \wedge \bot \equiv \bot,$$
$$p \vee \top \equiv \top, \qquad p \vee \bot \equiv p.$$

To verify that the optimizer preserves the semantics of formulas, several correctness results were formally proven in Coq.

```
Theorem optimizer_valid :
  forall V p, valid V p <-> valid V (optimizer p).

Theorem optimizer_satisfiable :
  forall p, satisfiable p <-> satisfiable (optimizer p).
```

To prove both theorems, `optimizer_eval_form` lemma was used, because at a certain point the hypothesis included `eval_form V p = true`, while the goal was `eval_form V (optimizer p) = true`. The lemma was proved by inducting on `p` and destructing `p1, p2` for `FAnd` and `FOr` cases. The other ones were trivially proved.

```
Lemma optimizer_eval_form :
  forall V p, eval_form V p = eval_form V (optimizer p).
```

Afterwards, it was pretty easy to prove `optimizer_valid`. For proving `optimizer_satisfiable`, initially I proved it in a very similar way as `solver_sound`. The proof seemed redundant and, after some time of thought, I was able to prove the theorem easier and cleaner by using `optimizer_valid` in the proof.

7 Extension 2: NNF Converter and Solver

The second extension implements a **Negation Normal Form (NNF) converter** for Boolean formulas. The purpose of this extension is to transform any formula into an equivalent one where negations only appear directly in front of variables and the only other allowed Boolean operators are conjunction and disjunction.

The core of the extension consists of two mutually recursive functions, `nnf_converter` – used for forms that should not be negated – and `nnf_negator` – used for the forms that should be negated.

I used these 2 mutually recursive functions because `Coq` does not allow something like this:

```
match p with
| FNot x    => match x with
  | FAnd x y => FAnd (nnf_converter (FNot x)) (nnf_converter (FNot y))
  | x => x
```

The theorem `nnf_converter_eval_form` proves the correctness of the converter, analogous to `optimizer_eval_form`. Because of the mutual recursive nature of the converter, the following lemma was needed for the proof.

```
Lemma negb_nnf_negator :
  forall V p,
  negb (eval_form V (nnf_converter p)) = eval_form V (nnf_negator p) /\
  negb (eval_form V (nnf_negator p))   = eval_form V (nnf_converter p).
```

Although only the first part of the lemma was used, the second one was needed for proving the lemma itself. Otherwise there was not enough context.

Once the NNF converter was verified, the `nnf_solver` was defined by simply applying the existing `solver` to the NNF-converted formula. The equivalence between `nnf_solver` and the standard `solver` is captured by the lemma `nnf_solver_eq`, which allows reusing the correctness of the original solver:

```
Lemma nnf_solver_eq p :
  nnf_solver p = solver (nnf_converter p).
```

Finally, using the above results, the soundness of the NNF-based solver was formally proven. The proof proceeds by rewriting `nnf_solver` in terms of `solver`, applying `solver_sound`, and using `nnf_converter_eval_form` to relate the original formula to its NNF form. This ensures that if `nnf_solver` returns true, a satisfying valuation for the original formula exists.

Notes

In this project, I attempted to use proof automation wherever possible by applying tactics such as `try` and systematically applying chaining tactics. This approach simplified the process of proving and made the proofs more structured and readable.